Currency exchange

Klevis Imeri

Goal 1. Design a deterministic $2^{k/2}$-competitive online algorithm $\mathcal{A}$ such that

$$\forall \sigma \in [1, 2^k]^T : \qquad c_{\mathcal{A}}(\sigma) \geq \frac{\mathrm{OPT}(\sigma)}{2^{k/2}},$$

and prove that no deterministic algorithm has a smaller competitive ratio.

Our idea is that if the exchange rate on a day is at least $2^{k/2}$, we exchange immediately. Otherwise, we wait, and if we have not exchanged before the last day, we exchange on day T.

To prove the bound for every input, we want to check what happens on a bad input. For example, all earlier rates could be below $2^{k/2}$, with the highest equal to $2^{k/2} - \varepsilon$, and the last rate could be 1. Then our algorithm picks 1, while the optimal offline algorithm picks $2^{k/2} - \varepsilon$, for a sufficiently small $\varepsilon > 0$. We use ε rather than 1 because the exchange rates need not be integers.

To prove the guarantee for every input sequence, we distinguish two cases.

1. Some rate reaches or exceeds the threshold $2^{k/2}$. Our algorithm exchanges on the first such day, so $c_{\mathcal{A}}(\sigma) \geq 2^{k/2}$. The exercise allows rates up to 2^k, so $\mathrm{OPT}(\sigma) \leq 2^k$. Thus,

   $$\frac{\mathrm{OPT}(\sigma)}{c_{\mathcal{A}}(\sigma)} \leq \frac{2^k}{2^{k/2}} = 2^{k/2}.$$

   This includes the case where the threshold is first reached on the last day. Then all earlier rates are below $2^{k/2}$, so the last rate is the maximum and

   $$c_{\mathcal{A}}(\sigma) = x_T = \mathrm{OPT}(\sigma), \qquad \frac{\mathrm{OPT}(\sigma)}{c_{\mathcal{A}}(\sigma)} = 1 \leq 2^{k/2}.$$

   The worst-case ratio $2^{k/2}$ is attained, for example, when $x_1 = 2^{k/2}$ and every later rate is 2^k.
2. No rate reaches the threshold $2^{k/2}$. Our algorithm exchanges on the last day and receives at least 1, while the optimal rate is below $2^{k/2}$. Thus,

   $$\frac{\mathrm{OPT}(\sigma)}{c_{\mathcal{A}}(\sigma)} < \frac{2^{k/2}}{1} = 2^{k/2}.$$

Therefore, for every input sequence σ,

$$c_{\mathcal{A}}(\sigma) \geq \frac{\mathrm{OPT}(\sigma)}{2^{k/2}},$$

so our deterministic algorithm is $2^{k/2}$-competitive.

Now we have to prove that this algorithm is optimal and no deterministic algorithm can do better.

The idea is to show that any deterministic algorithm must face a bad case, whatever choice it makes. In other words, we consider an arbitrary deterministic algorithm and ask whether we can choose future rates that make its current decision costly. This gives an adversarial lower bound.

Let $\mathcal{A}$ be any deterministic algorithm. On the first day, we give it the rate $x_1 = 2^{k/2}$. Now we have two cases. Every algorithm either exchanges on this day or waits.

1. If it exchanges on day 1, we set every remaining rate to 2^k. Then our algorithm receives $2^{k/2}$, while the offline optimum is 2^k, so
$$\frac{\mathrm{OPT}(\sigma)}{c_{\mathcal{A}}(\sigma)} = \frac{2^k}{2^{k/2}} = 2^{k/2}.$$
2. If it waits on day 1, we set every remaining rate to 1. Then it must eventually exchange at 1, while the offline optimum is $2^{k/2}$, so
$$\frac{\mathrm{OPT}(\sigma)}{c_{\mathcal{A}}(\sigma)} = \frac{2^{k/2}}{1} = 2^{k/2}.$$

Both sequences start with the same rate, so no online algorithm can distinguish between them on the first day. No matter what its first decision is, for every deterministic algorithm there is an input on which the ratio is at least $2^{k/2}$. Therefore, no deterministic algorithm has a smaller competitive ratio. □

Goal 2. Design a randomized $2k$-competitive online algorithm $\mathcal{A}$ such that
$$\forall \sigma \in [1, 2^k]^T : \qquad \mathbb{E}[c_{\mathcal{A}}(\sigma)] \geq \frac{\mathrm{OPT}(\sigma)}{2k}.$$

We use the same algorithm as above, but instead of fixing the threshold at $2^{k/2}$, we choose a threshold at random. The idea is that this limits the adversary's ability to use the same construction as in Goal 1. The adversary must fix the sequence independently of our random choices, so when choosing the first rate, it does not know which threshold we will use.